

MAST32001 Computational Statistics

Zhi Zhou

2026-09-20

```
knitr::opts_chunk$set(  
  echo = TRUE,  
  warning = FALSE,  
  message = FALSE,  
  fig.width = 5.5,  
  fig.height = 3.8,  
  fig.align = "center",  
  dpi = 300  
)  
library(ggplot2)  
theme_set(  
  theme_classic(base_size = 12, base_family = "serif") +  
    theme(plot.title = element_text(hjust = 0.5, face = "bold"))  
)
```

Exercise 1 — What may be permuted?

Task 1 - situation A — randomized experiment

The treatment labels can be shuffled because the group assignment was random. Each possible split into 12 treated and 12 control people was equally likely. Under the sharp null $Y_i(1) = Y_i(0) = Y_i$, each person's response stays the same whichever group they are assigned to.

Shuffling the labels repeats an assignment the experiment could have used. The responses stay fixed, and each group still has 12 people.

Task 2 - situation B — observational study, $H_0 : F_1 = F_2$

There is no random treatment assignment here, but we can still shuffle the labels under $H_0 : F_1 = F_2$. We assume that the 12 observations from each population are independent. Under the null, all 24 values come from the same distribution.

After pooling the values, the group labels no longer tell us anything about the distribution. Any split into two groups of 12 looks the same under the null. So shuffling labels is fine here.

Task 3 - situation C — why freely permuting all 24 labels is wrong

The pairs matter. Each person gives both a before and an after measurement, so the two values in a pair share a person effect. Taking the difference within a person removes that shared effect.

A shuffle across all 24 values would break that link. It would mix measurements from different people and no longer represent a comparison within the same 12 people.

Task 4 - a valid permutation scheme for C

We assume there is no before-after effect. Then the two values inside a pair are exchangeable. This holds independently for the 12 pairs. So we flip a coin inside each pair. Heads means keep “after minus before”. Tails means reverse it. The pairs themselves stay fixed. There are $2^{12} = 4096$ possible sign flips. Only the direction inside each pair changes.

Task 5 - what is permuted vs. what stays fixed

Situation	Permuted	Fixed
A randomized experiment	The treatment and control labels across the 24 people	Each person’s response and the two group sizes of 12
B observational, $F_1 = F_2$	The group labels across the 24 observations	The observed values and the two group sizes of 12
C matched pairs	The before and after labels within each person	The two measurements that make up each pair

Task 6 - why “nonparametric $\Rightarrow$ always shuffle” is wrong

“Nonparametric” does not mean “shuffle anything”. It just means we are not assuming a particular population shape. But we still need some condition to make shuffling valid.

In A, the labels can be shuffled because the assignment was random. In B, shuffling is valid when the observations are independent and come from the same distribution under the null. In C, we can swap the two values inside a pair if they are exchangeable under the null. But the pairs themselves must stay together.

Exercise 2 — What does the plot suggest?

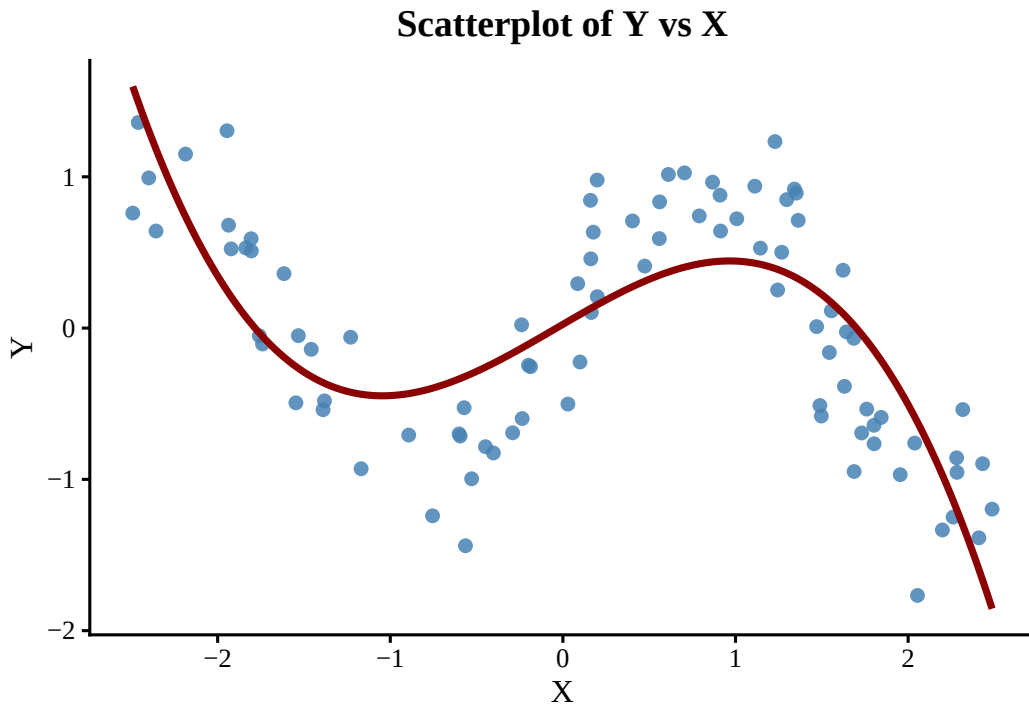
```
df <- read.table("Week3Ex2.txt", header = TRUE)
x <- df$x
y <- df$y
n <- length(x)
B <- 4999
```

Task 1 - scatterplot

```
xs <- data.frame(x = seq(min(x), (max(x)), length.out = 200))
xs$y <- predict(lm(y ~ x + I(x^2) + I(x^3), data = df), newdata = xs)

ggplot(df, aes(x, y)) +
  geom_point(
    color = "steelblue",
    size = 2.25,
    shape = 16,
    alpha = 0.85
  ) +
  geom_line(
    data = xs,
    aes(x, y),
    color = "darkred",
    linewidth = 1.25
  ) +
```

```
labs(x = "X", y = "Y", title = "Scatterplot of Y vs X") +
theme(
  axis.line = element_line(color = "black"),
  panel.grid = element_blank(),
  legend.position = c(0.85, 0.9)
)
```



The scatterplot of y against x does not show a straight line. Instead it shows a clear S-shaped curve.

Task 2 - observed correlations

Pearson's correlation measures a linear relationship. Spearman's correlation measures a trend that keeps the same direction. But neither one matches the two bends in the plot very well.

```
obs_pearson <- cor(x, y, method = "pearson")
obs_spearman <- cor(x, y, method = "spearman")
cat("n =", n, "\n")
cat("Observed Pearson r =", round(obs_pearson, 4), "\n")
cat("Observed Spearman rho =", round(obs_spearman, 4), "\n")
```

```
n = 90
Observed Pearson r = -0.3133
Observed Spearman rho = -0.3336
```

Both correlations are negative, but neither is large.

Task 3 - pearson permutation test

I keep x fixed and shuffle y 4,999 times. This breaks the observed links between the two variables. For independent observations, the shuffle is valid under the null that X and Y are independent.

```
set.seed(09190203)
perm_pearson <- vapply(seq_len(B), function(b)
  cor(x, sample(y), method = "pearson"), numeric(1))
pval_pearson <- (sum(abs(perm_pearson) >= abs(obs_pearson)) + 1) / (B + 1)
cat("Perm p (Pearson) =", round(pval_pearson, 4), "\n")
```

```
Perm p (Pearson) = 0.003
```

I compare the absolute observed correlation with the absolute correlations from the shuffled data:

$$p_{\text{Pearson}} = \frac{\#\{|r^*| \geq |r_{\text{obs}}|\} + 1}{B + 1} = 0.003.$$

The result gives evidence against independence at the 5% level.

Task 4 - spearman permutation test

I repeat the test with Spearman's correlation, again comparing absolute values.

```
set.seed(09190204)
perm_spearman <- vapply(seq_len(B), function(b)
  cor(x, sample(y), method = "spearman"), numeric(1))
pval_spearman <- (sum(abs(perm_spearman) >= abs(obs_spearman)) + 1) / (B + 1)
cat("Perm p (Spearman) =", round(pval_spearman, 4), "\n")
```

```
Perm p (Spearman) = 0.0024
```

The permutation p-value is 0.0024. This result also gives evidence against independence at the 5% level.

Task 5 - comparing the two p-values to the visual impression

Both tests find some relationship, which agrees with the plot. But a small p-value only says the variables are not independent. It does not tell us what shape the relationship has.

```
lin_r2 <- summary(lm(y ~ x))$r.squared
cat("Straight line model R^2 =", round(lin_r2, 4))
```

Straight line model $R^2 = 0.0982$

The straight-line model has $R\text{-squared} = 0.0982$, so it explains about 10% of the variation in y . A lot of the pattern is left over. Given the shape of the plot, a curved fit should do much better.

Task 6 - is the permutation procedure the problem, or something else?

The permutation procedure is not the problem. Keeping x fixed and shuffling y simulates independence correctly. The statistic is the problem. Pearson captures only a linear trend, and Spearman only a monotone trend. The S-curve changes direction, so both are pushed toward zero.

Task 7 - a statistic that better captures the relationship

I chose the cubic model ($y \sim x + x^2 + x^3$) mainly because the plot has two turns. The model uses all three terms together, so the fitted curve can change direction. Its R^2 tells how much of the variation in y the model explains.

Here, R^2 has no sign and ranges from 0 to 1. I fit the model again after each shuffle of y , then compare the new values with the observed R^2 .

```
# cubic fit R^2
set.seed(09190207)
cubic_r2 <- function(x, y)
  summary(lm(y ~ x + I(x^2) + I(x^3)))$r.squared
obs_r2 <- cubic_r2(x, y)
perm_r2 <- vapply(seq_len(B), function(b)
```

```

cubic_r2(x, sample(y)), numeric(1))
pval_r2 <- (sum(perm_r2 >= obs_r2) + 1) / (B + 1)
cat("Cubic R^2 observed =",
    round(obs_r2, 4),
    " (linear R2 =",
    round(lin_r2, 4),
    ")\n")
cat("Perm p (cubic R^2) =", round(pval_r2, 4), "\n")

```

```

Cubic R^2 observed = 0.5802 (linear R2 = 0.0982 )
Perm p (cubic R^2) = 2e-04

```

The cubic fit explains about 58% of the variation, compared with about 10% for the straight line. Its p-value is 0.0002, the smallest value this calculation can give with 4,999 shuffles.

Exercise 3 — Bootstrap uncertainty and the number of replicates

```

set.seed(091903)
x <- na.omit(airquality$Wind)
n <- length(x)

```

Task 1 - looking at the data

The wind speeds range from 1.7 to 20.7mph. The mean is about 10.0 mph, and the median is 9.7 mph. The standard deviation is about 3.5 mph.

```

summary(x)           # Min Q1 Median Mean Q3 Max
sd(x)
var(x)
df <- data.frame(wind = x)

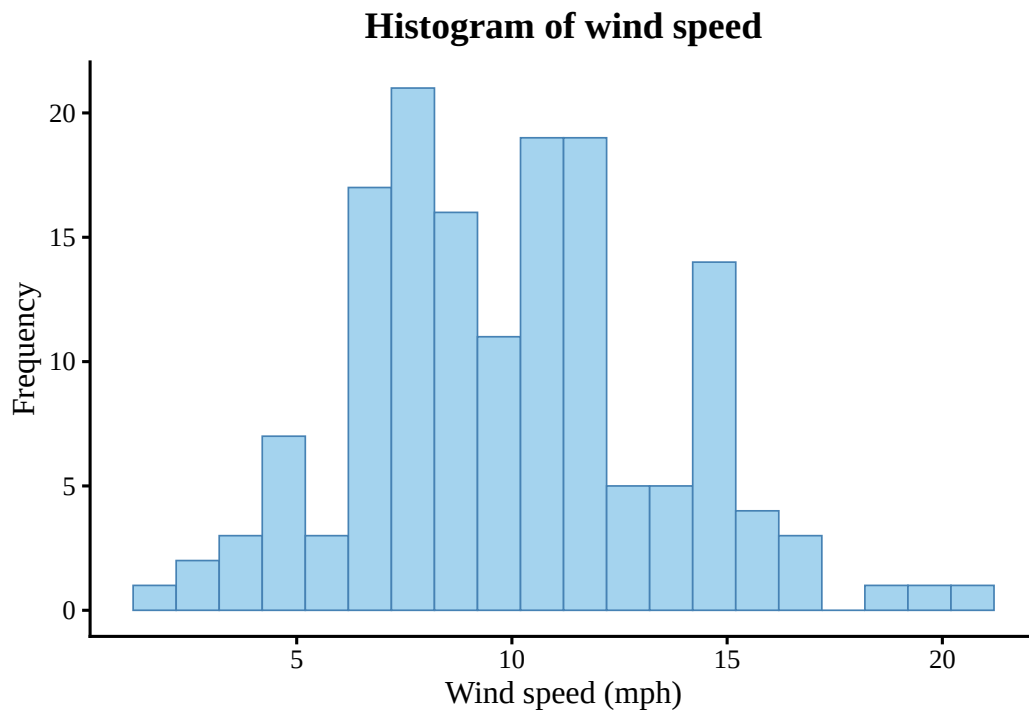
# histogram
ggplot(df, aes(x = wind)) +
  geom_histogram(
    bins = 20,

```

```

    fill = "lightskyblue2",
    color = "steelblue",
    linewidth = 0.3
  ) +
  labs(x = "Wind speed (mph)", y = "Frequency", title = "Histogram of wind speed") +
  theme(panel.grid = element_blank())

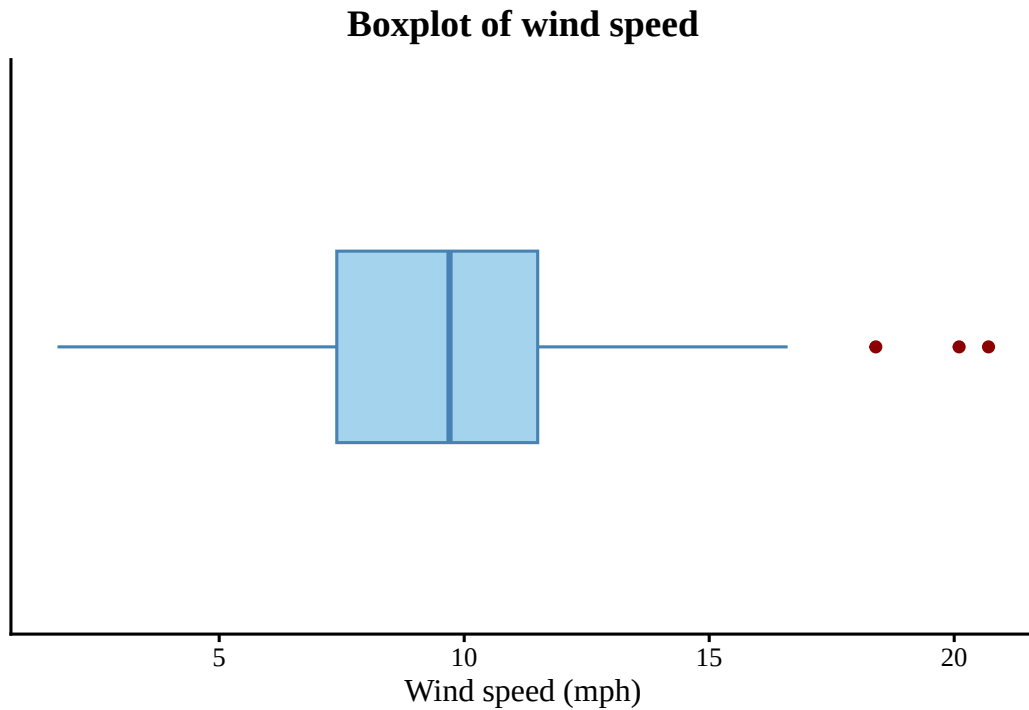
```



```

# horizontal boxplot
ggplot(df, aes(y = factor(1), x = wind)) +
  geom_boxplot(
    fill = "lightskyblue2",
    color = "steelblue",
    width = 0.4,
    outlier.color = "darkred"
  ) +
  labs(x = "Wind speed (mph)", y = NULL, title = "Boxplot of wind speed") +
  theme(
    panel.grid = element_blank(),
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank()
  )

```



Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.700	7.400	9.700	9.958	11.500	20.700

```
[1] 3.523001  
[1] 12.41154
```

The histogram has a longer tail on the right. The boxplot also shows a few high wind speeds beyond the upper whisker.

Task 2 - sample median

```
med_hat <- median(x)  
cat("Sample median wind speed =", round(med_hat, 3), "\n")
```

Sample median wind speed = 9.7

The median wind speed in this sample is 9.7 mph.

Task 3 - one nonparametric bootstrap example

I draw n values from the observed sample with replacement. I then calculate the median of this new sample.

```
boot_median <- function(x)
  median(sample(x, replace = TRUE))
boot_median(x)           # one bootstrap median
```

```
[1] 9.7
```

This bootstrap median is also 9.7 mph. It happens to match the original median.

Task 4 - bootstrap SE for each B

I estimate the standard error by the standard deviation of the B bootstrap medians. I repeat this for 100, 500, 2,000, and 10,000 resamples.

```
boot_se <- function(x, B) {
  s <- replicate(B, boot_median(x))
  sd(s)           # bootstrap SE
}
Bvals <- c(100, 500, 2000, 10000)
se_once <- sapply(Bvals, boot_se, x = x)
names(se_once) <- Bvals
print(round(se_once, 4))
```

```
      100      500     2000    10000
0.3555 0.3775 0.3657 0.3543
```

Task 5 - repeat the whole calculation 20 times per B

I repeat the full standard error calculation 20 times for each value of B . These runs show how much the estimate changes because of random resampling.

```

R    <- 20
res <- do.call(rbind, lapply(Bvals, function(b) {
  data.frame(B    = b,
             rep = seq_len(R),
             se   = replicate(R, boot_se(x, b)))
}))

```

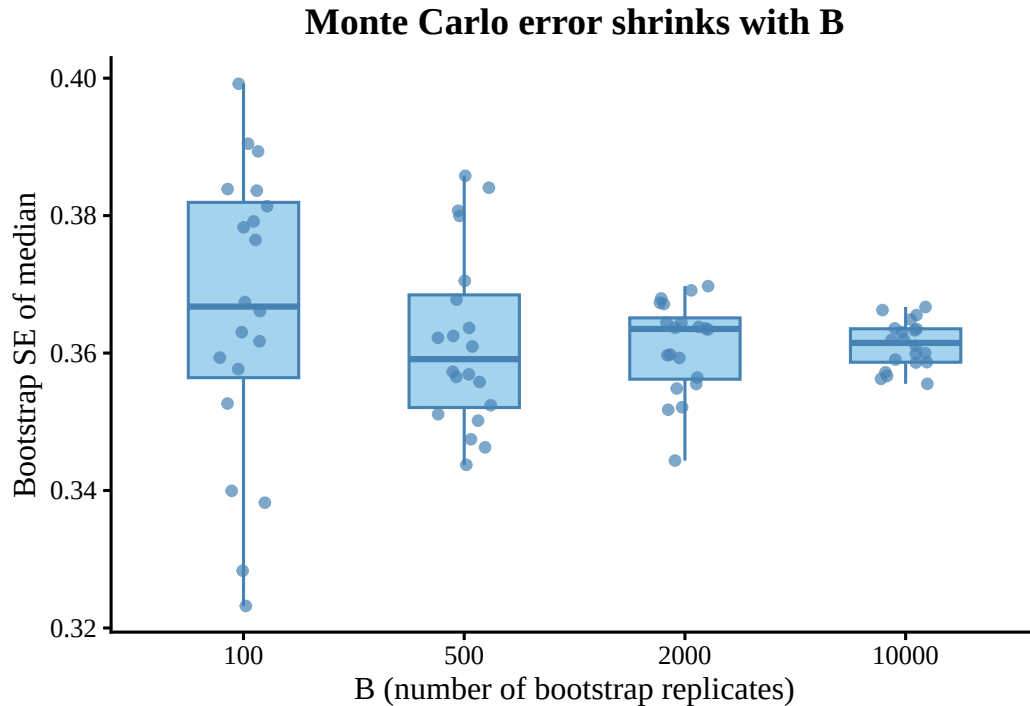
Task 6 - boxplot of the 20 SE estimates vs B

The boxplots compare the 20 standard error estimates from each setting.

```

res$B <- factor(res$B, levels = c(100, 500, 2000, 10000))
ggplot(res, aes(x = B, y = se)) +
  geom_boxplot(
    fill = "lightskyblue2",
    color = "steelblue",
    outlier.shape = NA,
    width = 0.5
  ) +
  geom_jitter(
    width = 0.12,
    size = 1.5,
    color = "steelblue",
    alpha = 0.7
  ) +
  labs(x = "B (number of bootstrap replicates)", y = "Bootstrap SE of median", title = "Monte C")
  theme(panel.grid = element_blank())

```



As B goes up, the 20 estimates get less spread out. Their centers stay about the same. So more resamples make the simulation more stable, but they do not reduce the real uncertainty from having only the observed sample.

Monte Carlo error usually decreases at about the rate $1/\sqrt{(B)}$. Increasing B from 100 to 10,000 reduces this simulation noise by about a factor of 10.

Task 7 - statistical uncertainty vs. Monte Carlo error

Statistical uncertainty comes from the sample size n . It says how far our estimate is from the truth. No choice of B removes it. Monte Carlo error is just the simulation noise from having only B resamples, and that one does go away as B grows.

Task 8 - why more B does not fix a small n

A larger B only reuses the same observations more times. If the original sample misses part of the population's pattern, the resamples will miss it too. So a stable bootstrap result can still be wrong.

Task 9 - is $B = 199$ enough to distinguish $p \approx 0.001$?

$B = 199$ is too small for this purpose. A bootstrap p-value has a resolution of about $1/(B + 1)$. For $B = 199$, that is

$$\frac{1}{199 + 1} = 0.005$$

With $B = 999$, the minimum is 0.001, but estimates near this minimum are very noisy. With $B = 9999$, the step size is 0.0001.

Exercise 4 — Confidence intervals for the median wind speed

```
theta_hat <- median(x)
B <- 10000
```

Task 1 - B bootstrap replicates of the median

I generate 10,000 bootstrap medians and use the same set for all three confidence intervals.

```
set.seed(092004)
boot_med <- replicate(B, median(sample(x, replace = TRUE)))
head(boot_med)
```

```
[1] 9.2 9.7 9.7 9.2 9.7 9.7
```

Task 2 - normal interval

The normal interval uses the sample median and the bootstrap standard error:

$$\hat{\theta} \pm 1.96 \hat{SE}_{boot}$$

```
se_boot <- sd(boot_med)
z <- qnorm(0.975)
ci_normal <- theta_hat + c(-1, 1) * z * se_boot
```

Task 3 - percentile interval

The percentile interval uses the 2.5th and 97.5th percentiles of the bootstrap medians:

$$[q_{0.025}^*, q_{0.975}^*].$$

```
ci_perc <- quantile(boot_med, c(0.025, 0.975))
```

Task 4 - basic interval

The basic interval reflects the bootstrap percentiles around the observed median:

$$[2\hat{\theta} - q_{0.975}^*, 2\hat{\theta} - q_{0.025}^*].$$

```
q_lo <- quantile(boot_med, 0.025)
q_hi <- quantile(boot_med, 0.975)
ci_basic <- 2 * theta_hat - c(q_hi, q_lo) # lower first: 2hat - q.975, 2hat - q.025

rbind(normal = ci_normal,
      percentile = ci_perc,
      basic = ci_basic)
cat("theta_hat =", theta_hat, " se_boot =", se_boot, "\n")
```

	2.5%	97.5%
normal	8.989598	10.4104
percentile	9.200000	10.3000
basic	9.100000	10.2000

theta_hat = 9.7 se_boot = 0.3624569

Task 5 - where the basic interval comes from

The bootstrap replaces $\hat{\theta} - \theta$ by $\hat{\theta}^* - \hat{\theta}$. The central 95% interval for this pivot is

$$q_{0.025}^* - \hat{\theta} \leq \hat{\theta} - \theta \leq q_{0.975}^* - \hat{\theta}.$$

We solve for θ . The right inequality gives

$$\theta \geq 2\hat{\theta} - q_{0.975}^*.$$

The left inequality gives

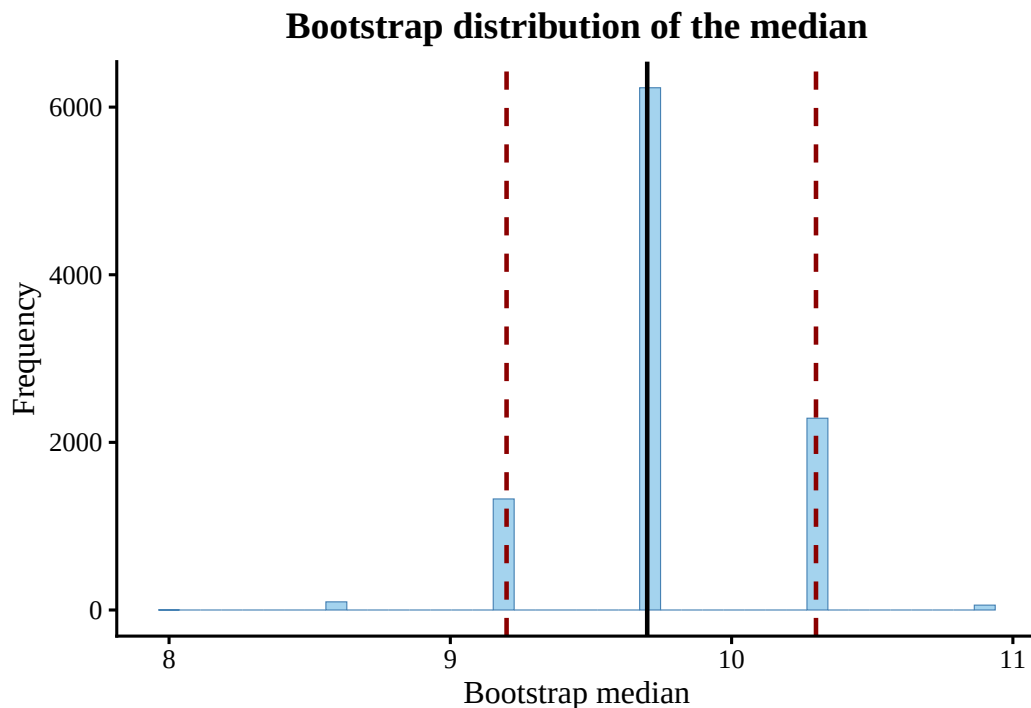
$$\theta \leq 2\hat{\theta} - q_{0.025}^*.$$

The term $2\hat{\theta}$ appears for a simple reason. We move $\hat{\theta}$ across the inequalities, and two copies remain. So the basic interval is just the percentile interval mirrored around $\hat{\theta}$.

Task 6 - plot the bootstrap distribution

The histogram code uses the same bootstrap medians as the interval calculations. The black line marks the sample median, and the red dashed lines mark the percentile limits.

```
df <- data.frame(b = boot_med)
ggplot(df, aes(x = b)) +
  geom_histogram(
    bins = 40,
    fill = "lightskyblue2",
    color = "steelblue",
    linewidth = 0.2
  ) +
  geom_vline(xintercept = theta_hat,
    color = "black",
    linewidth = 0.8) +
  geom_vline(
    xintercept = c(q_lo, q_hi),
    color = "darkred",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  labs(x = "Bootstrap median", y = "Frequency", title = "Bootstrap distribution of the median",
  theme(panel.grid = element_blank())
```



Task 7 - comparing the three intervals

The normal interval is widest in these results and is centered on 9.7 mph. The percentile and basic intervals have the same width, but the basic interval is 0.1 mph lower. This shift comes from reflecting the percentile limits around the sample median.

The percentile and basic intervals match only if the two limits are equally far from $\hat{\theta}$. That does not always happen. The normal interval is also close when the bootstrap distribution is roughly normal and centered there. Symmetry alone does not make all three intervals equal.

Task 8 - colleague claims the median is 10 mph

All three reported intervals contain 10 mph. We would not reject a population median of 10 mph using the tests that match these intervals. This conclusion is at the approximate two-sided 5% level. It does not prove that the population median is exactly 10 mph.

Task 9 - “There is a 95% probability the true median is between my endpoints”

The 95% refers to how the method works over repeated samples. The true median is fixed, but the interval changes with the sample. If the method works as intended, about 95% of the intervals

from repeated samples will contain the true median. For the interval we have already calculated, the true median is either inside it or outside it. We do not assign a 95% probability to that fixed value.

Task 10 - a value just outside the interval

A value outside a 95% confidence interval is rejected by the two-sided 5% test that corresponds to that interval. This link requires the interval and the test to use the same method.

For continuous methods, a value just outside the interval often gives a p-value close to 0.05. Bootstrap distributions can be discrete, so the p-value may change in steps. So if we only have the interval, we cannot read off an exact p-value.

Exercise 5 — When the ordinary bootstrap fails

```
set.seed(092005)
n <- 30
theta <- 0
x <- runif(n, theta, 1)
m_obs <- min(x)
m_obs
```

```
[1] 0.01102128
```

Task 1 - $B = 5000$ ordinary bootstrap minima

The sample contains 30 observations from $U(0,1)$. Its minimum is about 0.011. I draw 5,000 bootstrap samples and calculate the minimum of each sample.

```
B <- 5000
boot_min <- replicate(B, min(sample(x, replace = TRUE)))
```

Task 2 - $m_n^* \geq m_n$ for every bootstrap sample

```
all(boot_min >= m_obs)      # TRUE, by construction: no resample can go below
mean(boot_min == m_obs)    # fraction exactly at m_n ≈ 1 - (1-1/n)^n ≈ 0.64
```

```
[1] TRUE
```

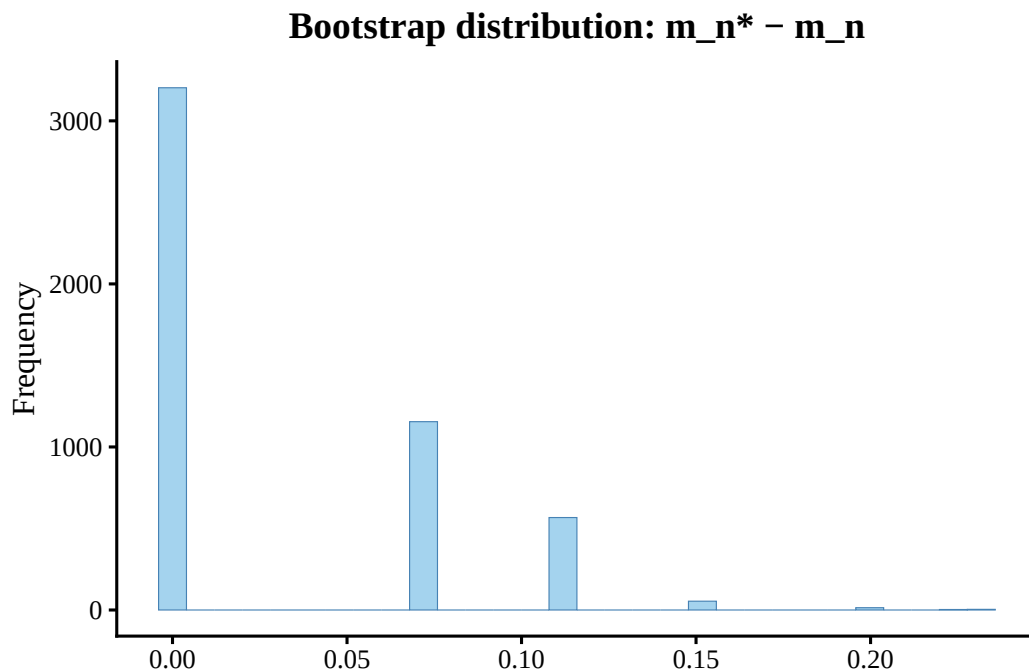
```
[1] 0.6406
```

Every resample satisfies $m_n^* \geq m_n$. In this run, 64.06% of the bootstrap minima equal the observed minimum.

Task 3 - bootstrap distribution of $m_n^* - m_n$

The histogram shows $m_n^* - m_n$. All differences are zero or positive. The large bar at zero comes from resamples that contain the original minimum.

```
ggplot(data.frame(d = boot_min - m_obs), aes(x = d)) +
  geom_histogram(
    bins = 30,
    fill = "lightskyblue2",
    color = "steelblue",
    linewidth = 0.2
  ) +
  labs(x = "", y = "Frequency", title = "Bootstrap distribution: m_n* - m_n") +
  theme(panel.grid = element_blank())
```



Task 4 - 5000 fresh samples from the TRUE $U(0, 1)$

I draw 5,000 fresh samples from $U(0, 1)$, with 30 values in each sample. I calculate each sample minimum to estimate its true sampling distribution.

```
true_min <- replicate(B, min(runif(n, theta, 1)))
```

Task 5 - compare to the true sampling distribution $m_n^{new} - \theta$

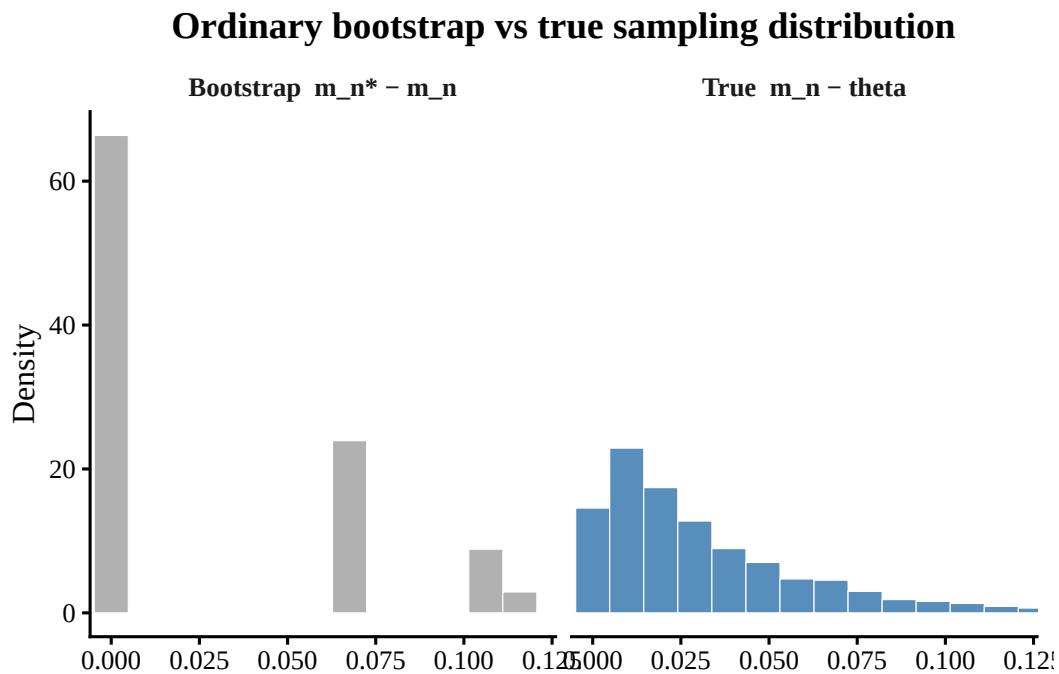
I compare the bootstrap errors $m_n^* - m_n$ with the fresh-sample errors $m_n^{new} - \theta$. The bootstrap errors have a large probability mass at zero. The true errors are continuous and have no point mass at zero. So the bootstrap histogram has a spike at 0, but the true one does not.

```
cmp <- rbind(
  data.frame(diff = boot_min - m_obs, src = "Bootstrap m_n* - m_n"),
  data.frame(diff = true_min - theta, src = "True m_n - theta")
)
ggplot(cmp, aes(x = diff, fill = src)) +
  geom_histogram(
    aes(y = after_stat(density)),
```

```

bins = 25,
color = "white",
linewidth = 0.2,
alpha = 0.9,
show.legend = FALSE
) +
scale_fill_manual(values = c(
  "Bootstrap  $m_n^* - m_n$ " = "darkgrey",
  "True  $m_n - \theta$ " = "steelblue"
)) +
facet_wrap(~ src, nrow = 1) +
coord_cartesian(xlim = c(0, 0.12)) +
labs(x = "", y = "Density", title = "Ordinary bootstrap vs true sampling distribution") +
theme(
  panel.grid = element_blank(),
  strip.background = element_blank(),
  strip.text = element_text(face = "bold")
)

```



Task 6 - why F_n cannot reproduce the lower-endpoint uncertainty

The empirical distribution only knows the observed values. Since θ is below minimum, there is no data there. So no resample can draw a value below the observed minimum. The bootstrap cannot tell us anything about the chance that θ is smaller.

Task 7 - contrast with the sample mean

The mean uses all the observations, so many values contribute to its variation. For independent observations from the same distribution, a finite, nonzero variance gives an approximately normal sampling distribution in large samples. The ordinary bootstrap can estimate the mean's uncertainty under these conditions.

The minimum depends on the single lowest value. In this uniform example, its uncertainty depends on the gap between that value and the lower endpoint. Resampling the observed values does not reproduce that gap correctly.

Task 8 - another statistic where the ordinary bootstrap fails

The sample range, $\max(X_i) - \min(X_i)$, has the same kind of problem in this uniform example. A resample stays within the observed minimum and maximum, so its range cannot be larger than the observed range.

```
boot_range <- replicate(B, {
  z <- sample(x, replace = TRUE)
  max(z) - min(z)
})
obs_range <- max(x) - min(x)
true_range <- replicate(B, {
  u <- runif(n)
  max(u) - min(u)
})

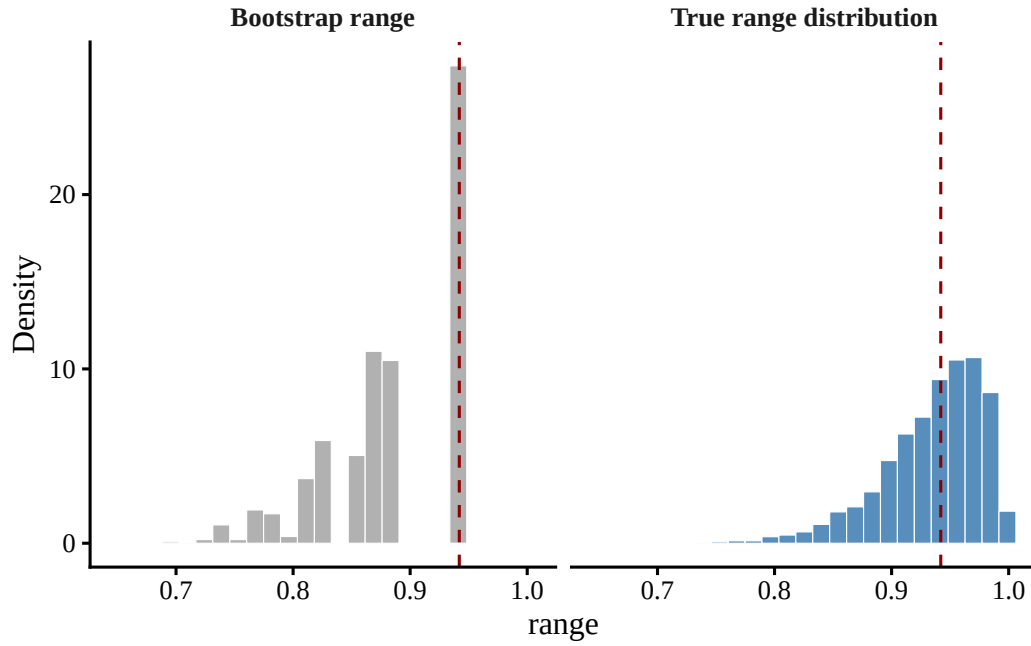
cmp2 <- rbind(
  data.frame(r = boot_range, src = "Bootstrap range"),
  data.frame(r = true_range, src = "True range distribution")
)
ggplot(cmp2, aes(x = r, fill = src)) +
```

```

geom_histogram(
  aes(y = after_stat(density)),
  bins = 25,
  color = "white",
  linewidth = 0.2,
  alpha = 0.9,
  show.legend = FALSE
) +
geom_vline(xintercept = obs_range,
           color = "darkred",
           linetype = "dashed") +
scale_fill_manual(values = c(
  "Bootstrap range" = "darkgrey",
  "True range distribution" = "steelblue"
)) +
facet_wrap( ~ src, nrow = 1) +
labs(x = "range", y = "Density", title = "Range: bootstrap truncated at the observed range")
theme(
  panel.grid = element_blank(),
  strip.background = element_blank(),
  strip.text = element_text(face = "bold")
)

```

Range: bootstrap truncated at the observed range



About 40% of resamples contain both original extremes, so they give exactly the same range. Fresh samples have continuous ranges and can land on either side of the observed one. The repeated value and the upper limit in the bootstrap distribution give the wrong picture of the uncertainty. The sample maximum has a similar problem at the upper endpoint.